Crime / Narrative NLI Pipeline

Complete Project Explanation — File by File

What this project does: Given a **claim** about a book (e.g. "Holmes used a telescope"), the system automatically checks whether that claim is **consistent** with the book or **contradicts** it. It does this using a Retrieval-Augmented Generation (RAG) pipeline combined with a from-scratch Transformer NLI model.

Files Covered in This Document

File	Role
`config.py`	Central settings — all hyperparameters in one place
`data_loader.py`	Loads books (.txt) and CSV files from disk
`chunking.py`	Breaks books into overlapping 200-word passages
`embedding.py`	Converts passages to dense vectors using MiniLM
`retrieval.py`	Hybrid BM25 + semantic search for relevant passages
`dataset_builder.py`	Combines passages + claims into model input strings
`custom_nli_model.py`	From-scratch Transformer NLI architecture
`train_scratch.py`	Full training loop with R-Drop, focal loss, early stopping
`inference.py`	Threshold optimisation + test set predictions

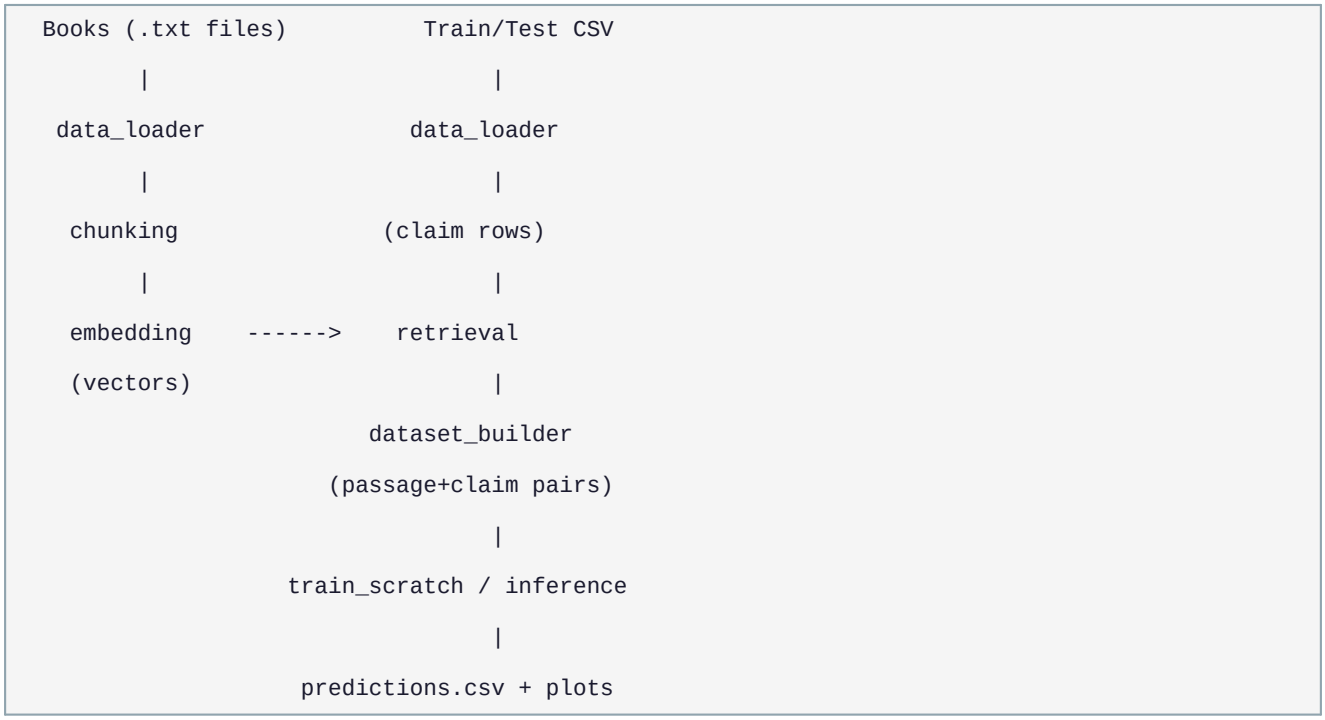
Overall Pipeline Architecture

Two Main Components

The project has two core components working together as a **RAG pipeline** (Retrieval-Augmented Generation):

COMPONENT 1: RETRIEVAL SYSTEM	COMPONENT 2: NLI MODEL
Find the most relevant parts of the book for this claim	Given those relevant parts + the claim, predict consistent or contradict

End-to-End Data Flow



1. config.py — Central Settings

Purpose: The single source of truth for the entire project. Every other file imports constants from here. Change a value here and the whole pipeline adapts — no hunting through multiple files.

Data Paths

```
BOOKS_DIR  = 'Dataset/Books'             # folder with .txt book files
TRAIN_CSV  = 'Dataset/combined_train_v2.csv'  # labelled training data
TEST_CSV   = 'Dataset/test.csv'          # claims to predict (no labels)
MODEL_SAVE_DIR   = 'model/'              # where trained weights are saved
PREDICTIONS_PATH = 'predictions.csv'     # final output file
```

Model & Retrieval Settings

```
EMBEDDING_MODEL_NAME = 'sentence-transformers/all-MiniLM-L6-v2'
# Used to convert text into vectors for similarity search

TOP_K         = 3    # retrieve 3 passages per claim
CHUNK_SIZE    = 200  # each passage = 200 words
CHUNK_OVERLAP = 50   # consecutive passages share 50 words
```

Training Hyperparameters

```
BATCH_SIZE    = 16       # samples processed at once
NUM_EPOCHS    = 25       # maximum training rounds
LEARNING_RATE = 2.5e-4  # how fast model learns
MAX_SEQ_LEN   = 256      # max tokens per input
SEED          = 42       # for reproducibility
```

Label Mapping

```
LABEL2ID = {'consistent': 1, 'contradict': 0}
ID2LABEL = {0: 'contradict',  1: 'consistent'}
# Model works with numbers internally
# We convert strings -> numbers before training
# And numbers -> strings for final output
```

2. data_loader.py — Loading Raw Data

Purpose: The very first step. Opens files from disk and loads them into Python. Three functions handle books, training data, and test data separately.

Function 1: load_books()

Reads every **.txt** file from the Books folder and returns a dictionary.

```
Input:  Dataset/Books/sherlock_holmes.txt
        Dataset/Books/dracula.txt

Output: {
  'sherlock_holmes': 'It was in the year 1878...',
  'dracula':         '3 May. Bistritz...',
}

Key = filename without .txt (lowercase)
Value = entire book as one string
```

Function 2: load_train_csv()

Reads the labelled training CSV and drops empty rows.

```
CSV structure:
  id | book_name       | content                    | label
  1  | sherlock_holmes | Holmes used a telescope    | contradict
  2  | dracula         | Van Helsing came at night  | consistent

After loading:
  - Drops rows where content or label is empty
  - Returns clean DataFrame ready for use
```

Function 3: load_test_csv()

Same as train but **no label column** — these are the claims you need to predict.

```
Smart feature: auto-detects whether header row exists
(some test files have headers, some don't)

Output schema: id | book_name | content
  (no label column — that's what we predict)
```

3. chunking.py — Breaking Books Into Pieces

Purpose: A full book has 100,000+ words — too large to search directly. This file breaks each book into small overlapping chunks (passages) that can be individually searched and retrieved.

Why Overlapping Chunks?

```
CHUNK_SIZE = 200 words, CHUNK_OVERLAP = 50 words

Step size  = 200 - 50 = 150 words

Chunk 0:  words   0 -> 199
Chunk 1:  words 150 -> 349   <- overlaps with chunk 0 by 50 words
Chunk 2:  words 300 -> 499   <- overlaps with chunk 1 by 50 words

WHY? If a key sentence sits at words 195-205, without overlap
it gets split across two chunks and relevant context is lost.
Overlap ensures every sentence appears FULLY in at least one chunk.
```

Output: Passage Objects

```
Passage(
    book_name = 'sherlock_holmes',
    chunk_id  = 42,
    text      = 'Holmes crossed the room...'  # exactly 200 words
)

A 100,000 word book -> ~667 passages
All books together  -> several thousand passages total
```

Two Functions

- **chunk_book(book_name, text)** — chunks a single book
- **chunk_all_books(books)** — calls chunk_book for every book, returns flat list

4. embedding.py — Converting Text to Vectors

Purpose: Computers cannot compare text directly. This file converts every passage into a list of 384 numbers (a vector) that captures its meaning. Passages about similar things will have similar vectors — enabling fast similarity search.

What an Embedding Is

```
'Holmes picked up the magnifying glass'
              | MiniLM model
[0.23, -0.41, 0.87, 0.12, -0.33, ...]   <- 384 numbers

'The detective examined the lens carefully'
              | MiniLM model
[0.21, -0.38, 0.84, 0.15, -0.29, ...]   <- similar numbers!

Same meaning -> similar numbers
Different meaning -> different numbers
```

The Embedding Process

```
Step 1: Take all passage chunks from all books
Step 2: Feed each chunk's text through MiniLM model
Step 3: L2-normalise every vector (length = 1.0)
        -> cosine similarity = simple dot product (much faster)
Step 4: Stack all vectors into matrix of shape (N, 384)
        e.g. 5000 passages -> matrix shape (5000, 384)
```

Caching System

```
FIRST RUN:  encode all passages (takes a few minutes)
            save to cache/embedding_index/
              embeddings.npy  <- the actual vectors
              passages.pkl    <- passage objects
              meta.json       <- model name, count

LATER RUNS: load from cache instantly (seconds)

--rebuild-index flag: forces fresh encoding
  Use when you change books or chunk settings
```

Output: EmbeddingIndex

```
EmbeddingIndex(
    passages   = [Passage(...), Passage(...), ...],
    embeddings = numpy array shape (N, 384),
    model_name = 'all-MiniLM-L6-v2'
)
```

5. retrieval.py — Finding Relevant Passages

Purpose: Given a claim, find the most relevant passages from the correct book. Uses two search methods combined — semantic search catches paraphrases, BM25 keyword search catches exact matches. Together they cover each other's weaknesses.

Method 1: Semantic Search (60% weight)

```
Step 1: Encode claim as 384-dim vector using MiniLM
Step 2: Compute dot product vs ALL passage vectors
        (fast because vectors are L2-normalised)

passage_0_score = claim_vec . passage_0_vec = 0.43
passage_1_score = claim_vec . passage_1_vec = 0.87  <- high!
passage_2_score = claim_vec . passage_2_vec = 0.31

Good at: different words, same meaning (paraphrases)
Weakness: misses specific numbers, names, exact terms
```

Method 2: BM25 Keyword Search (40% weight)

```
Claim: 'Holmes counted fifteen steps'
Tokens: ['holmes', 'counted', 'fifteen', 'steps']

For each passage: count how often claim keywords appear
Rare words score HIGHER than common words
  'fifteen' is rare -> big score boost when it matches
  'the'     is common -> tiny score even when it matches

Good at: exact matches — character names, numbers, terms
Weakness: misses paraphrases entirely
```

Hybrid Score Formula

```
final_score = 0.6 x semantic_score + 0.4 x BM25_score

Example: Claim 'Holmes counted fifteen steps'
  semantic score for correct passage: 0.72
  BM25 score for correct passage:     0.89  <- 'fifteen' exact match!
  hybrid score: 0.6x0.72 + 0.4x0.89 = 0.432 + 0.356 = 0.788

Pure semantic only: 0.72  |  Pure BM25 only: 0.89
```

```
Hybrid: 0.788 <- balanced and robust
```

Book Filter

Only passages from the **correct book** are returned. If the claim is about Sherlock Holmes, Dracula passages are never returned.

6. dataset_builder.py — Building Model Inputs

Purpose: Combines retrieved passages and the claim into a single string that the NLI model can process. This is the bridge between retrieval and the model.

The Input String Format

```
CLAIM: 'Holmes used a telescope to observe the suspect'
BOOK:  sherlock_holmes

RETRIEVAL -> Top 3 passages:
  Passage A: 'Holmes stood at the window. He raised his hand...'
  Passage B: 'The detective used various instruments...'
  Passage C: 'Watson noted that Holmes preferred to observe...'

COMBINED INPUT STRING:
  'Passage A text [SEP] Passage B text [SEP] Passage C text [SEP] claim'
   ^-- passages come first (evidence)                    claim comes last --^
```

The [SEP] Token

```
[SEP] = separator token between sections

The tokenizer assigns token_type_ids:
  token_type_id = 0 -> passage tokens (the evidence)
  token_type_id = 1 -> claim tokens (what to verify)

This helps the model understand WHICH part is evidence
and WHICH part is the claim being checked
```

Output: ClassificationSample

```
ClassificationSample(
    sample_id  = 1,
    input_text = 'passage1 [SEP] passage2 [SEP] passage3 [SEP] claim',
    label      = 0,   # 0=contradict, 1=consistent, -1=test (unknown)
    book_name  = 'sherlock_holmes'
)
```

7. custom_nli_model.py — The Neural Network

Purpose: Defines the Transformer NLI model built completely from scratch using PyTorch. No pretrained weights from BERT or RoBERTa — every weight starts random and is learned purely from your book data. ~10.7 million parameters total.

Layer 1: Embedding

```
Three learned embedding tables combined:

TOKEN EMBEDDING:   word-piece ID -> 256 numbers
                   'holmes' -> [0.12, -0.34, 0.87, ...]

POSITION EMBEDDING: position 0 -> [0.01, 0.22, -0.15, ...]
                    position 1 -> [0.34, -0.11, 0.45, ...]
                    Tells model WHERE each token is in sequence

SEGMENT EMBEDDING:  segment 0 (passage) -> [0.11, -0.22, ...]
                    segment 1 (claim)   -> [-0.15, 0.41, ...]
                    Tells model: is this passage or claim?

FINAL = sum of all three -> LayerNorm -> Dropout
Output shape: (batch, sequence_length, 256)
```

Layer 2: 4 Transformer Blocks (Self-Attention + FFN)

```
SELF-ATTENTION (8 heads):
  Every token attends to every other token
  Q, K, V = linear projections of each token
  score(i, j) = Q_i . K_j / sqrt(256)
  output_i = weighted sum of all V vectors
  8 heads -> each learns different relationships

FEED-FORWARD NETWORK:
  Linear(256->512) -> GELU -> Linear(512->256)

RESIDUAL CONNECTIONS (pre-LayerNorm):
  x = x + attention(LayerNorm(x))
  x = x + ffn(LayerNorm(x))
  Prevents vanishing gradients, enables depth
```

Layer 3: Dual Pooling

```
After 4 transformer blocks, output shape: (batch, seq_len, 256)
Need to compress to single vector per sample.

CLS TOKEN:   first token vector = global sequence meaning
             cls_vec = output[:, 0, :]   shape: (batch, 256)

MEAN POOL:   average all real tokens (ignore padding)
             mean_vec = sum(tokens) / count(real_tokens)
             shape: (batch, 256)

COMBINED:    concatenate([cls_vec, mean_vec])
             shape: (batch, 512)

WHY BOTH? CLS captures overall relationship.
Mean pool captures distributed token-level evidence.
Together = richer representation than either alone.
```

Layer 4: Classification Head (MLP)

```
Input: 512-dim pooled vector

512 -> Linear -> 1024 -> GELU -> Dropout -> LayerNorm
1024 -> Linear -> 128 -> GELU -> Dropout
128 -> Linear -> 2   (logits: contradict / consistent)

softmax([logit0, logit1]) -> [prob_contradict, prob_consistent]
e.g. [0.73, 0.27] = 73% confident it is a contradiction
```

Focal Loss

```
Normal cross-entropy: all examples contribute equally
PROBLEM: easy examples dominate; model lazy on hard cases

Focal Loss:
  Easy example (90% confident): loss x (1-0.9)^2 = loss x 0.01
  Hard example (51% confident): loss x (1-0.51)^2 = loss x 0.24
  -> Forces model to focus on difficult, ambiguous cases

Label Smoothing (0.15):
  Hard targets [0, 1] -> Soft targets [0.075, 0.925]
  Prevents overconfidence, improves probability calibration
```

9. inference.py — Making Predictions

Purpose: Runs the trained model on test data and finds the optimal decision threshold. The threshold determines how aggressively the system flags contradictions — critical for a crime novel consistency checker.

Threshold Optimisation

```
MODEL OUTPUT for each sample:
  P(contradict) = 0.37
  P(consistent) = 0.63

DEFAULT RULE: if P(consistent) >= 0.50 -> consistent

BUT: missing a real contradiction (saying 'consistent' when
it's 'contradict') is WORSE than a false alarm.
So maybe threshold = 0.40 is better?

THRESHOLD SWEEP:
  For each threshold in [0.30, 0.31, ..., 0.79, 0.80]:
    Apply to val set probabilities
    score = (macro_F1 + contradiction_recall) / 2

  threshold=0.50: F1=0.82, recall=0.75, score=0.785
  threshold=0.45: F1=0.81, recall=0.83, score=0.820
  threshold=0.40: F1=0.79, recall=0.91, score=0.850 <- best!
  threshold=0.35: F1=0.74, recall=0.95, score=0.845

Pick threshold=0.40
```

Prediction on Test Set

```
PROCESS IN BATCHES OF 16:

  For each batch:
    1. Tokenize: passage [SEP] claim -> token IDs
    2. Forward pass through model
    3. Softmax -> [prob_contradict, prob_consistent]
    4. Apply threshold: P(consistent) >= 0.40? -> consistent
                                               -> contradict

COLLECT all predictions -> ['consistent', 'contradict', ...]
```

Output: predictions.csv

```
predictions.csv:
  id  | label
  1   | contradict
  2   | consistent
  3   | consistent
  4   | contradict
  ...

Note: main.py runs inference TWICE
  First with default threshold=0.50
  Then with optimal threshold (overwrites the file)
```

Three Functions in inference.py

Function	What it does
`find_optimal_threshold()`	Sweeps 0.30-0.80, returns best threshold
`predict()`	Runs model on test samples, returns label list
`save_predictions()`	Pairs predictions with IDs, writes predictions.csv

Complete Pipeline — Quick Reference

File Responsibilities at a Glance

Step	File	Input	Output
1	`config.py`	—	Settings constants
2	`data_loader.py`	.txt + CSV files	books dict + DataFrames
3	`chunking.py`	books dict	list of Passage objects
4	`embedding.py`	Passage list	EmbeddingIndex (vectors)
5	`retrieval.py`	EmbeddingIndex + claim	Top-K RetrievedPassages
6	`dataset_builder.py`	CSV rows + retriever	ClassificationSamples
7	`custom_nli_model.py`	—	NarrativeNLIModel class
8	`train_scratch.py`	train ClassificationSamples	Trained model + history
9	`inference.py`	test ClassificationSamples	predictions.csv

Command Line Usage

```
# Full run: train from scratch + evaluate + save plots
python3 main.py

# Skip training, reuse already-saved model
python3 main.py --skip-training

# Force rebuild of embedding index
python3 main.py --rebuild-index

# Both flags together
python3 main.py --skip-training --rebuild-index
```

Key Design Decisions:

- **Hybrid retrieval** (BM25 + semantic) covers both exact matches and paraphrases
- **From-scratch model** learns exactly what matters for book consistency checking
- **R-Drop + focal loss** combat overfitting and class imbalance on small datasets
- **Dual pooling** (CLS + mean) gives richer representations than CLS alone
- **Threshold optimisation** prioritises catching contradictions over precision
- **Two-tier checkpointing** saves both better classifiers and better-calibrated models